



Universidad Tecnológica de Panamá Facultad
de Ingeniería en Sistemas Computacionales
Licenciatura en Ciberseguridad



Materia

Programación II

Profesor

Napoleon Ibarra

Estudiante

Stefany Guerrero 4-833-2081

Trabajo

Investigación I

Fecha

27/3/2026

Semestre I

I Parte. Investigación 1

Origen e Historia

Python fue creado por el programador holandés Guido van Rossum en 1989, durante sus vacaciones de Navidad, mientras trabajaba en el Centro de Matemáticas e Informática (CWI) en los Países Bajos. Guido buscaba un lenguaje que corrigiera las deficiencias de ABC, un lenguaje educativo en el que había participado, y que fuera fácil de usar, potente y con una sintaxis clara.

La primera versión, Python 0.9.0, se publicó en 1991 y ya incluía clases con herencia, manejo de excepciones, funciones y tipos de datos fundamentales como listas, diccionarios y cadenas de texto.

En el año 2000 llegó Python 2.0, que introdujo características importantes como la comprensión de listas y un recolector de basura cíclico. Sin embargo, el hito más significativo en su historia ocurrió en 2008 con el lanzamiento de Python 3.0, una versión que rompió la compatibilidad con versiones anteriores para corregir errores de diseño y limpiar el lenguaje. Esta decisión, aunque polémica en su momento, permitió que Python evolucionara de manera más sólida.

Durante muchos años, Guido van Rossum lideró el desarrollo con el título de "Benevolente Dictador Vitalicio" (BDFL), hasta que en 2018 anunció su retiro, dejando la gobernanza del lenguaje en manos de la Python Software Foundation (PSF), una organización sin fines de lucro que continúa guiando su evolución.

Concepto

Python es un lenguaje de programación de alto nivel, interpretado, de tipado dinámico y multiparadigma, diseñado con una filosofía que prioriza la legibilidad y la simplicidad del código. Su nombre no proviene de la serpiente, sino del grupo de comedia británico Monty Python, del cual su creador era fanático.



Características Principales de Python

La popularidad de Python se debe a una combinación única de características que lo hacen poderoso y accesible al mismo tiempo.

Característica	Descripción	Beneficio Principal
Sintaxis simple y legible	El código se lee como si fuera inglés. Usa sangrías, no llaves.	Fácil de aprender y entender.
Interpretado	El código se ejecuta línea por línea, sin compilar.	Pruebas rápidas y depuración sencilla.
Tipado dinámico	No declaras el tipo de la variable al crearla. Solo asignas un valor y Python lo detecta solo.	Escribes menos código, más rápido, y la misma variable puede cambiar de tipo si le asignas otro valor después.
Multiparadigma	Puedes programar de varias formas: con clases, con funciones, o mezclado.	Te adaptas al problema, no al revés.
Baterías incluidas	Viene con muchas herramientas ya listas (archivos, internet, matemáticas).	No necesitas instalar cosas extra para empezar.
Código abierto	Es gratis y cualquiera puede usarlo o mejorarlo.	Tiene una comunidad enorme que ayuda y comparte.
Multiplataforma	Funciona en Windows, Mac, Linux, etc.	Haces el código una vez y corre en todos lados.

Importancia y Relevancia de Python

1. Líder en inteligencia artificial y ciencia de datos

Es el lenguaje principal en machine learning e IA gracias a bibliotecas como TensorFlow, PyTorch, Pandas y NumPy.

2. Fácil aprendizaje

Su sintaxis clara lo convierte en el lenguaje más usado en universidades y cursos para principiantes.

3. Alta demanda laboral

Es uno de los lenguajes más solicitados por empresas, con perfiles bien remunerados como desarrollador Python, científico de datos e ingeniero de machine learning.

4. Usado por grandes empresas

Google, Netflix, Instagram, Spotify, NASA y bancos confían en Python para sus sistemas críticos.

5. Automatización y productividad

Permite automatizar tareas repetitivas de forma sencilla, siendo útil incluso para no programadores.

6. Comunidad gigante

Cuenta con millones de desarrolladores, abundante soporte y más de 500,000 bibliotecas disponibles.

7. Versátil

Sirve para desarrollo web, ciencia de datos, IA, automatización, aplicaciones de escritorio y más.

8. Impulsa la innovación

Ha democratizado tecnologías avanzadas, permitiendo avances en medicina, biotecnología, educación y finanzas.

II Parte. Investigación 2

Cuadro Comparativo: Listas vs Tuplas en Python

Aspecto	Lista	Tupla
Definición	Colección ordenada y mutable de elementos. Puede contener cualquier tipo de dato.	Colección ordenada e inmutable de elementos. Puede contener cualquier tipo de dato.
Sintaxis	Se escribe con corchetes [] Ejemplo: [1, 2, 3]	Se escribe con paréntesis () Ejemplo: (1, 2, 3)
Mutabilidad	Si se puede modificar después de creada. Puedes cambiar, agregar o quitar elementos libremente.	No se puede modificar después de creada. Una vez que defines la tupla, queda fija para siempre.
Agregar elementos	Con <code>append()</code> (agrega al final) o <code>insert()</code> (agrega en posición específica).	No existe el método <code>append()</code> ni <code>insert()</code> en tuplas. Si intentas agregar, Python te mostrará un error.
Eliminar elementos	Con <code>remove()</code> (elimina por valor) o <code>pop()</code> (elimina por índice).	No existen los métodos <code>remove()</code> ni <code>pop()</code> en tuplas. No puedes quitar ningún elemento.

Cambiar elementos	Asignando un nuevo valor directamente: lista[0] = "nuevo valor".	No puedes reasignar valores mediante índice. tupla[0] = "nuevo" genera un error TypeError.
Métodos disponibles	append(), extend(), insert(), remove(), pop(), sort(), reverse(), copy(), clear()	Solo count() (cuenta cuántas veces aparece un valor) e index() (encuentra la posición de un valor). No tiene métodos que modifiquen la estructura.
Velocidad	Más lenta porque Python necesita prepararse para posibles cambios en el tamaño o contenido.	Más rápida porque Python sabe que la estructura nunca cambiará, lo que permite optimizaciones.
Uso de memoria	Mayor consumo porque reserva espacio extra para futuras adiciones o modificaciones.	Menor consumo porque ocupa exactamente el espacio necesario para sus elementos, sin reservas adicionales.
Se puede usar como clave de diccionario	No. Las listas no son "hashable" porque son mutables. Si una lista cambia, su identificador también cambiaría, lo que rompería el diccionario.	Si. Las tuplas son "hashable" porque son inmutables. Su valor nunca cambia, por lo que son seguras como claves en diccionarios.
Aplicabilidad (cuándo usarla)	Usa lista cuando: Los datos cambian durante la ejecución del programa, Necesitas agregar o quitar elementos con frecuencia, Trabajas con colecciones dinámicas como listas de usuarios, carritos de compra, resultados de búsqueda, etc.	Usa tupla cuando: Los datos son fijos y no deben cambiar nunca, Trabajas con coordenadas, días de la semana, colores primarios, configuraciones fijas, Necesitas usarlas como claves en diccionarios, Buscas mejor rendimiento en lecturas repetidas y menor consumo de memoria

Cuadro Comparativo: Conjuntos vs Diccionarios en Python

Aspecto	Conjunto (set)	Diccionario (dict)
Concepto	Colección desordenada de elementos únicos. No permite elementos	Colección desordenada de pares clave-valor. Cada

	repetidos. Cada elemento aparece una sola vez.	clave es única y se asocia a un valor.
Sintaxis	Se escribe con llaves { } o con set() Ejemplo: {1, 2, 3}	Se escribe con llaves { } y dos puntos entre clave y valor Ejemplo: {"nombre": "Ana", "edad": 25}
Estructura	Almacena solo elementos (valores sueltos). No hay pares ni relaciones.	Almacena pares clave-valor. Cada clave tiene un valor asociado.
Métodos principales	add(), remove(), union(), intersection()	keys(), values(), items(), get(), pop()
Elementos repetidos	No permite duplicados. Si agregas un elemento que ya existe, simplemente se ignora.	No permite claves duplicadas. Si usas una clave repetida, el nuevo valor sobrescribe al anterior. Los valores sí pueden repetirse.
Uso de memoria	Menor consumo porque solo almacena valores.	Mayor consumo porque almacena claves y valores.
z	Usa conjunto cuando: Necesitas asegurar que no haya elementos duplicados. Quieres hacer operaciones matemáticas como unión, intersección o diferencia entre colecciones. Solo te interesa saber si un elemento está presente o no. No necesitas asociar información adicional a cada elemento	Usa diccionario cuando: Necesitas asociar un valor a una clave (relación uno a uno). Quieres organizar información con etiquetas o identificadores. Necesitas búsquedas rápidas por clave. Trabajas con datos estructurados como perfiles de usuarios, configuraciones, respuestas de API

Métodos para la manipulación de listas en Python

Métodos Principales

Acción	Método	Descripción
Agregar al final	append(x)	Añade un elemento x al final de la lista
Agregar varios elementos	extend(iterable)	Añade todos los elementos de un iterable al final
Eliminar por índice y obtener valor	pop(i)	Elimina y devuelve el elemento en la posición i. Si

		no se especifica, elimina el último
Eliminar por valor	remove(x)	Elimina la primera aparición del elemento x. Si no existe, genera error
Buscar posición	index(x)	Devuelve el índice de la primera aparición de x. Si no existe, genera error
Ordenar	sort()	Ordena la lista de forma ascendente. Con reverse=True ordena descendente

Métodos Secundarios

Acción	Método	Descripción
Agregar en posición específica	insert(i, x)	Inserta x en la posición i, desplazando los elementos a la derecha
Contar apariciones	count(x)	Cuenta cuántas veces aparece x en la lista
Invertir	reverse()	Invierte el orden de los elementos de la lista
Copiar	copy()	Devuelve una copia superficial de la lista
Eliminar todos	clear()	Elimina todos los elementos y deja la lista vacía

Términos por definir

Término Explicación sencilla

Alto nivel Es un lenguaje que se parece más al idioma humano que al lenguaje de la computadora. No necesitas saber cómo funciona el procesador para programar.

Interpretado El código se ejecuta línea por línea, no necesita convertirse completo antes de correr. Esto permite probar rápido y encontrar errores fácilmente.

Término	Explicación sencilla
Tipado dinámico	No tienes que decirle a Python qué tipo de dato es una variable. Solo le asignas un valor y él solito lo entiende. Por ejemplo, puedes poner <code>x = 5</code> y después <code>x = "hola"</code> sin problema.
Multiparadigma	Puedes programar de diferentes formas según lo que necesites: con funciones, con clases, o mezclando estilos. No te obliga a usar una sola manera.
Baterías incluidas	Python ya trae muchas herramientas listas para usar sin instalar nada extra. Por ejemplo, trabajar con archivos, fechas, matemáticas, etc.
Código abierto	El código de Python es público y gratis. Cualquiera puede verlo, usarlo y mejorarlo.
Multiplataforma	Un programa hecho en Python funciona igual en Windows, Mac, Linux y otros sistemas operativos sin cambiarlo.
Mutable	Algo que se puede cambiar después de creado. Como una lista: puedes agregar, quitar o modificar sus elementos.
Inmutable	Algo que no se puede cambiar después de creado. Como una tupla: una vez que la creas, queda fija para siempre.
Hashable	Un objeto que tiene un valor único que no cambia. Solo los objetos inmutables (como tuplas) pueden ser hashable, por eso solo ellas pueden ser claves de diccionarios.
Iterable	Cualquier cosa que se pueda recorrer elemento por elemento, como una lista, una tupla o un conjunto.
Colección	Un grupo de datos guardados juntos. Por ejemplo: listas, tuplas, conjuntos y diccionarios son colecciones.

Término	Explicación sencilla
Clave-valor	Una forma de organizar datos donde cada información tiene una etiqueta (clave) y un contenido (valor). Como en un diccionario real: "nombre" es la clave y "Ana" es el valor.
Elementos únicos	En un conjunto no puede haber elementos repetidos. Si intentas agregar el mismo valor dos veces, solo se guarda una vez.